



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Bueno Hernández Héctor Daniel

N° de Cuenta: 319233148

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2025-2

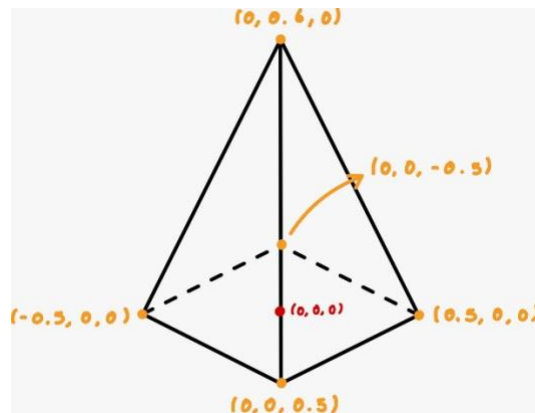
FECHA DE ENTREGA LÍMITE: 29/03/2025

CALIFICACIÓN: _____

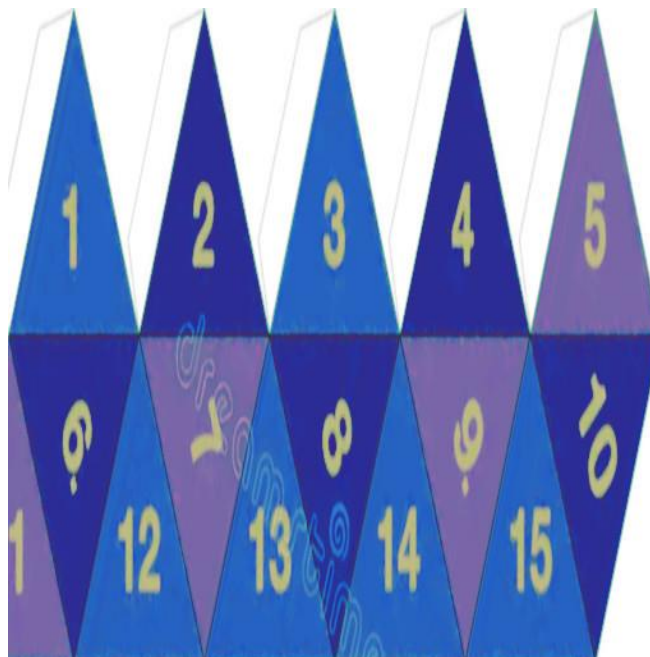
Dado de 8 caras

Se solicitó realizar el modelo de un dado de 8 caras por medio de código en OpenGL, el cual debe encontrarse debidamente texturizado. Para ello, lo primero que se realizó fue un boceto de la pirámide para poder calcular más fácilmente las coordenadas de cada uno de los vértices.

Si bien el dado es un polígono de 8 caras, este está compuesto de dos pirámides cuadrangulares. Debido a esto, solamente fue necesario obtener las coordenadas de la pirámide superior ya que lo único que varía entre ambas pirámides es la coordenada de la punta:



Una vez que se establecieron los vértices para crear el dado, se modificó la imagen correspondiente a la textura en GIMP. La imagen se recortó para solamente mantener la parte útil de esta y se escaló a 512x512 píxeles. Finalmente, se exportó en formato .jpg y el resultado fue el siguiente:



Para agregar la textura al dado, se especificaron las coordenadas de cada uno de los vértices de la imagen a los vértices establecidos en el código usando las coordenadas S y T. Esto fue sencillo debido a que la imagen se puede dividir en cinco segmentos horizontales y dos verticales, en los cuales se encuentran contenidos los triángulos de las caras.

Recordando que las coordenadas S y T van de izquierda a derecha en el eje S y de abajo para arriba en el eje T, los vértices generados del dado ya con la textura fueron los siguientes:

```
void CrearDado()
{
    unsigned int indices_piramide_triangular[] = {
        //top pyramid
        0, 1, 2,
        3, 4, 5,
        6, 7, 8,
        9, 10, 11,
        //bottom pyramid
        12, 13, 14,
        15, 16, 17,
        18, 19, 20,
        21, 22, 23,
    };

    GLfloat vertices_piramide_triangular[] = {
        //x      y      z      S      T      NX      NY      NZ
        //top front
        -0.5f,  0.0f,  0.5f,  0.01f,  0.51f,  0.0f,  0.0f, -1.0f, //0
        0.5f,   0.0f,  0.5f,  0.19f,  0.51f,  0.0f,  0.0f, -1.0f, //1
        0.0f,   0.65f,  0.0f,  0.10f,  0.99f,  0.0f,  0.0f, -1.0f, //2
        //top back
        -0.5f,  0.0f, -0.5f,  0.59f,  0.51f,  0.0f,  0.0f,  1.0f, //0
        0.5f,   0.0f, -0.5f,  0.41f,  0.51f,  0.0f,  0.0f,  1.0f, //1
        0.0f,   0.65f,  0.0f,  0.50f,  0.99f,  0.0f,  0.0f,  1.0f, //2
        //top right
        0.5f,   0.0f,  0.5f,  0.21f,  0.51f, -1.0f,  0.0f,  0.0f, //0
        0.5f,   0.0f, -0.5f,  0.39f,  0.51f, -1.0f,  0.0f,  0.0f, //1
        0.0f,   0.65f,  0.0f,  0.30f,  0.99f, -1.0f,  0.0f,  0.0f, //2
        //top left
        -0.5f,  0.0f,  0.5f,  0.79f,  0.51f,  1.0f,  0.0f,  0.0f, //0
        -0.5f,  0.0f, -0.5f,  0.61f,  0.51f,  1.0f,  0.0f,  0.0f, //1
        0.0f,   0.65f,  0.0f,  0.70f,  0.99f,  1.0f,  0.0f,  0.0f, //2

        //bottom front
        -0.5f,  0.0f,  0.5f,  0.50f,  0.01f,  0.0f,  0.0f, -1.0f, //0
        0.5f,   0.0f,  0.5f,  0.41f,  0.49f,  0.0f,  0.0f, -1.0f, //1
        0.0f,   -0.65f,  0.0f,  0.59f,  0.49f,  0.0f,  0.0f, -1.0f, //2
        //bottom back
        -0.5f,  0.0f, -0.5f,  0.01f,  0.49f,  0.0f,  0.0f,  1.0f, //0
        0.5f,   0.0f, -0.5f,  0.10f,  0.01f,  0.0f,  0.0f,  1.0f, //1
        0.0f,   -0.65f,  0.0f,  0.19f,  0.49f,  0.0f,  0.0f,  1.0f, //2
        //bottom right
        0.5f,   0.0f,  0.5f,  0.99f,  0.51f, -1.0f,  0.0f,  0.0f, //0
        0.5f,   0.0f, -0.5f,  0.81f,  0.51f, -1.0f,  0.0f,  0.0f, //1
        0.0f,   -0.65f,  0.0f,  0.90f,  0.99f, -1.0f,  0.0f,  0.0f, //2
        //bottom left
        -0.5f,  0.0f,  0.5f,  0.21f,  0.49f,  1.0f,  0.0f,  0.0f, //0
        -0.5f,  0.0f, -0.5f,  0.30f,  0.01f,  1.0f,  0.0f,  0.0f, //1
        0.0f,   -0.65f,  0.0f,  0.39f,  0.49f,  1.0f,  0.0f,  0.0f, //2
    };

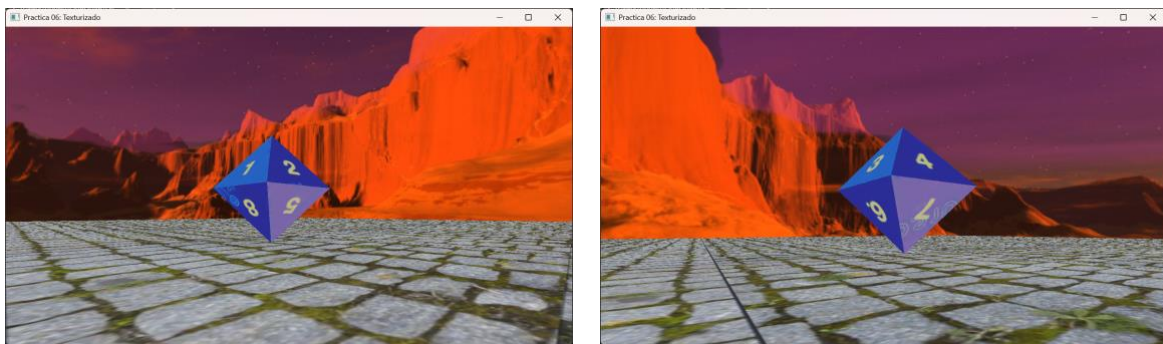
    Mesh* dado = new Mesh();
    dado->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 192, 24);
    meshList.push_back(dado);
}
```

Para crear la instancia del dado, el código es bastante similar al que se ha estado usando durante las prácticas anteriores. Primero se declara una matriz y a esta se le aplican las transformaciones que se requieran, después, para indicar que el dado va a utilizar una textura, se debe incluir la línea `dadoTexture.UseTexture()`, donde dicha función ya fue declarada anteriormente y contiene la dirección de la imagen de la textura.

Los fragmentos de código generados fueron los siguientes:

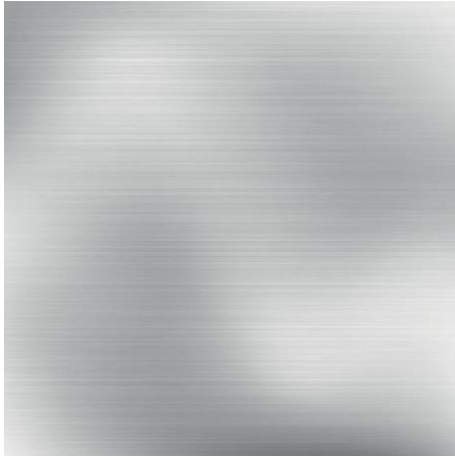
```
Texture brickTexture;  
Texture dirtTexture;  
Texture plainTexture;  
Texture pisoTexture;  
Texture dadoTexture;  
Texture logofiTexture;  
  
brickTexture = Texture("Textures/brick.png");  
brickTexture.LoadTextureA();  
dirtTexture = Texture("Textures/dirt.png");  
dirtTexture.LoadTextureA();  
plainTexture = Texture("Textures/plain.png");  
plainTexture.LoadTextureA();  
pisoTexture = Texture("Textures/piso.tga");  
pisoTexture.LoadTextureA();  
dadoTexture = Texture("Textures/textura_dado.jpg");  
dadoTexture.LoadTextureA();  
logofiTexture = Texture("Textures/escudo_fi_color.tga");  
logofiTexture.LoadTextureA();  
  
//Dado de 8 caras  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
dadoTexture.UseTexture();  
meshList[4]->RenderMesh();
```

Al ejecutar el código, el dado se ve de la siguiente forma:

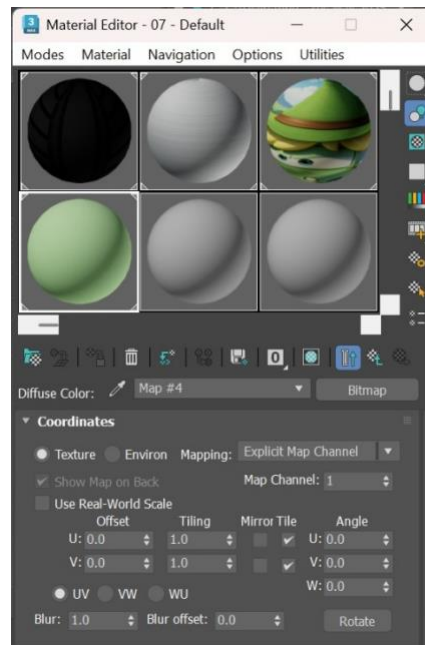


Texturizado de las llantas

Utilizando el modelo del carro con el que se trabajó en la práctica anterior, se solicitó agregar textura a las llantas de este para diferenciar el caucho de los rines. Para ello, primero se obtuvo una imagen de textura metálica y otra de la textura de una llanta para poder crear los materiales dentro de 3ds Max. Estas fueron escaladas en GIMP y se exportaron en formato .jpg:



Después, se crearon los materiales en el *material editor* utilizando un *bitmap* para poderlos colocar en los modelos. En la siguiente imagen, los materiales correspondientes a la llanta son los primeros dos que se muestran:



Una vez que se texturizó el modelo de la llanta, este se exportó en formato .obj cuidando las configuraciones para que las texturas también se exportaran adecuadamente.

Ya dentro del código, se declaró el objeto de la clase `Model` y se inicializó para poder especificar la ruta de donde se encuentra el modelo de la llanta:

```
Model Elantra_M;  
Model Llanta_M;  
Model Dado_M;
```

```
Elantra_M = Model();  
Elantra_M.LoadModel("Models/Coche/elantra.obj");  
Llanta_M = Model();  
Llanta_M.LoadModel("Models/Coche/llanta.obj");  
Dado_M = Model();  
Dado_M.LoadModel("Models/dadoanimales.obj");
```

Para poder instanciar las llantas, se siguió la misma estructura que con dado en el sentido que se aplicaron transformaciones (*translate*, *rotate* y *scale*) para después mandar a llamar el modelo antes declarado. En este caso, las llantas se están colocando de forma jerárquica, donde el chasis del carro es el modo padre, por lo que se usa una variable auxiliar llamada `modelaux` para no heredar las transformaciones a las demás llantas.

Los fragmentos de código generados para cada llanta son los siguientes:

```
//Llanta delantera izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-3.0f, -0.8f, 1.75f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

//Llanta trasera izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(3.025f, -0.8f, 1.75f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

//Llanta delantera derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-3.0f, -0.8f, -1.75f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

//Llanta trasera derecha
model = modelaux;
model = glm::translate(model, glm::vec3(3.025f, -0.8f, -1.75f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();
```

Texturizado del carro

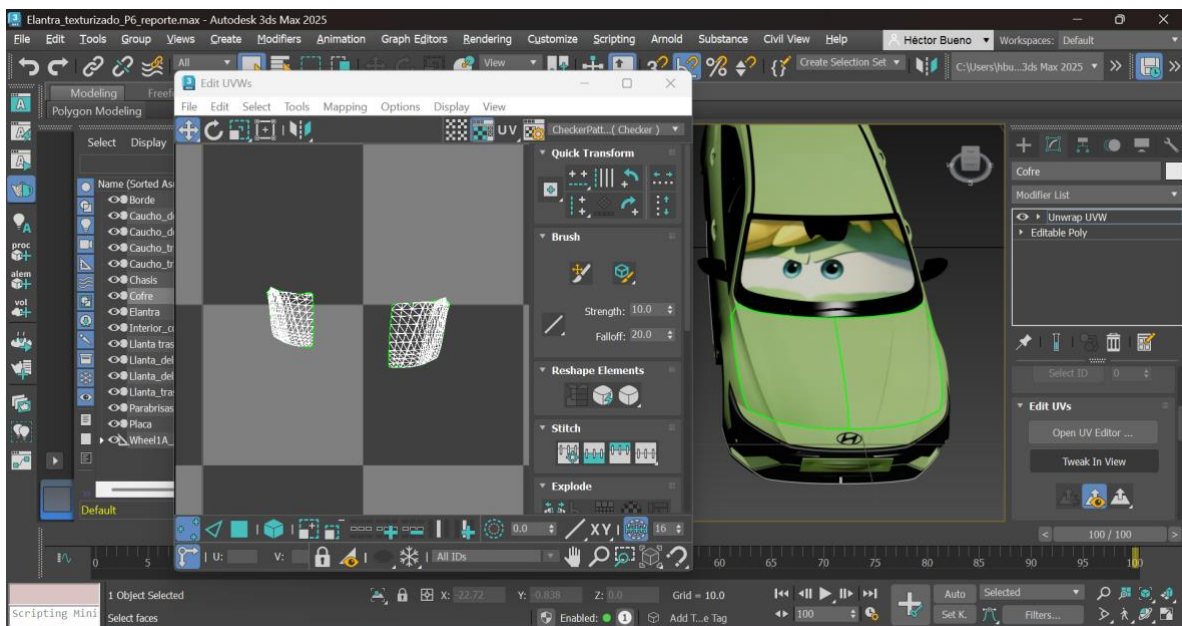
Continuando con la aplicación de texturas al modelo del carro, tomando como referencia el carro en estilo “cars” realizado con inteligencia artificial del personaje elegido para el proyecto final.

Yo elegí el personaje Link de The Legend of Zelda: Wind Waker y tuve que modificar levemente la imagen seleccionada originalmente por una en la que se puede observar un poco mejor el cofre y el parabrisas con los ojos del personaje. La imagen utilizada es la siguiente:

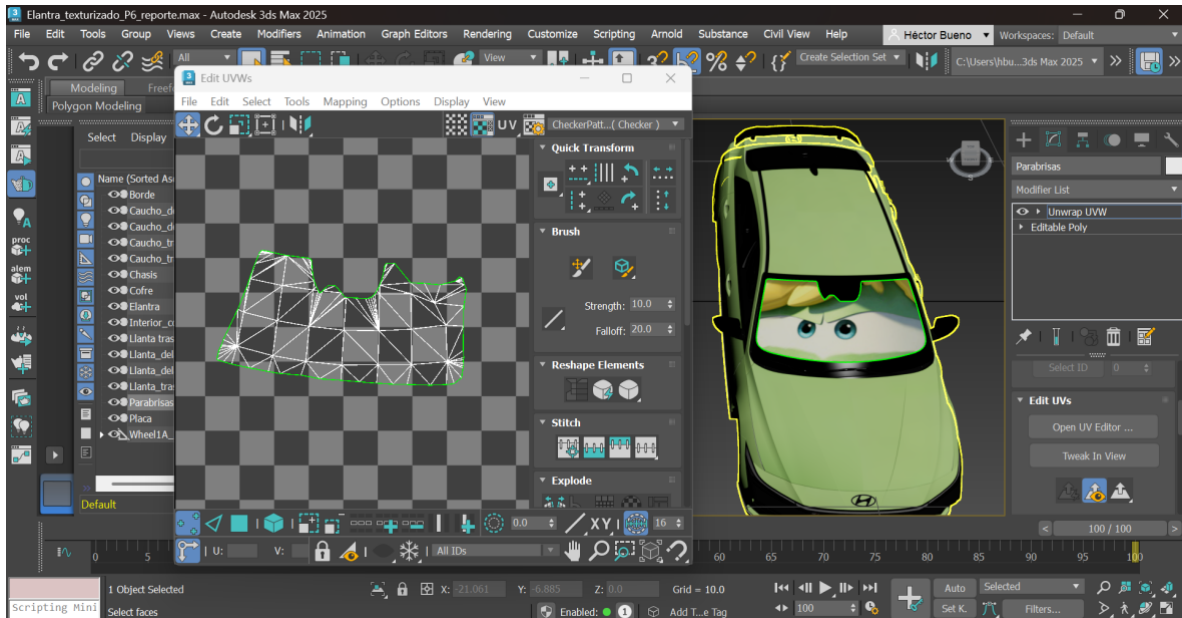


Para poder aplicar la textura a los modelos dentro de 3ds Max, era necesario que los modelos del cofre, el parabrisas y el chasis se encontraran por separado. Después de llevar a cabo el procedimiento correspondiente, se hizo uso de *UVWs Editor*. Dentro de este, se escalaron y se modificó la posición de los vértices para que la textura embonara adecuadamente.

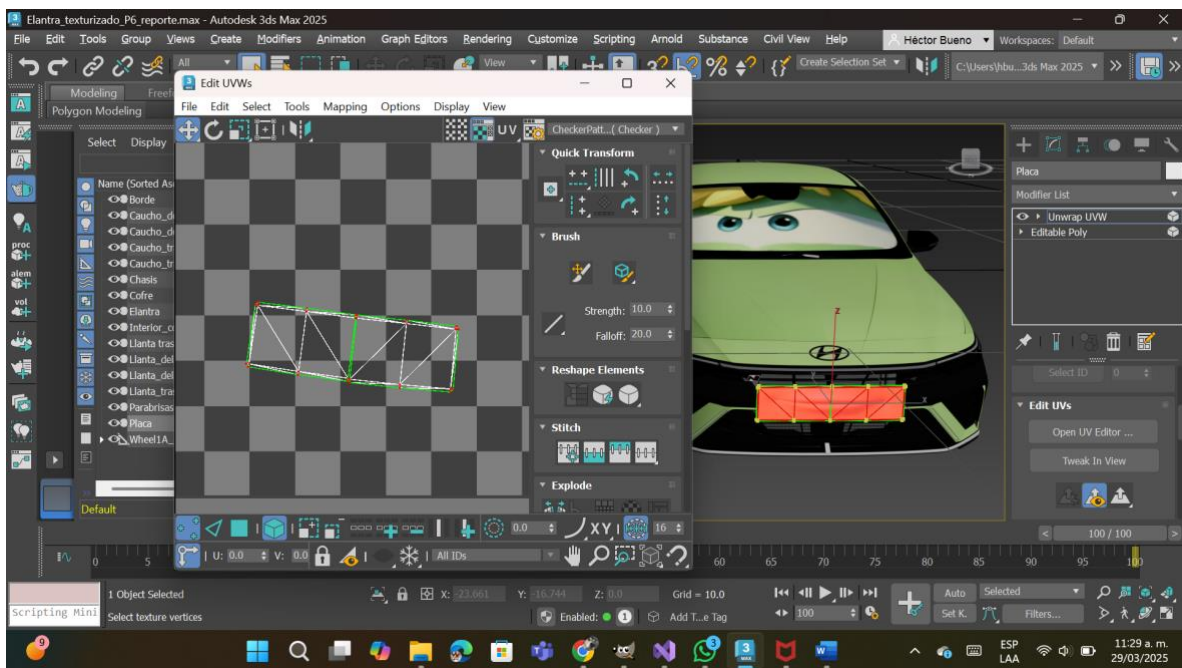
En el caso de cofre, este es de color verde sólido, por lo que solamente se hicieron pequeñas las partes del cofre y se colocaron en una parte del cofre del dibujo que no contara con sombras o dibujos. El resultado dentro del editor fue el siguiente:



Para el parabrisas, se movieron por individual todos vértices de la orilla para que la forma de este se adecuara al parabrisas de la imagen de referencia. Debido q que en la imagen el parabrisas no se encuentra de frente a la cámara, la forma resultante en el editor se ve muy deformada, sin embargo, gracias a esto se puede apreciar correctamente los ojos en el modelo del carro como se muestra a continuación:



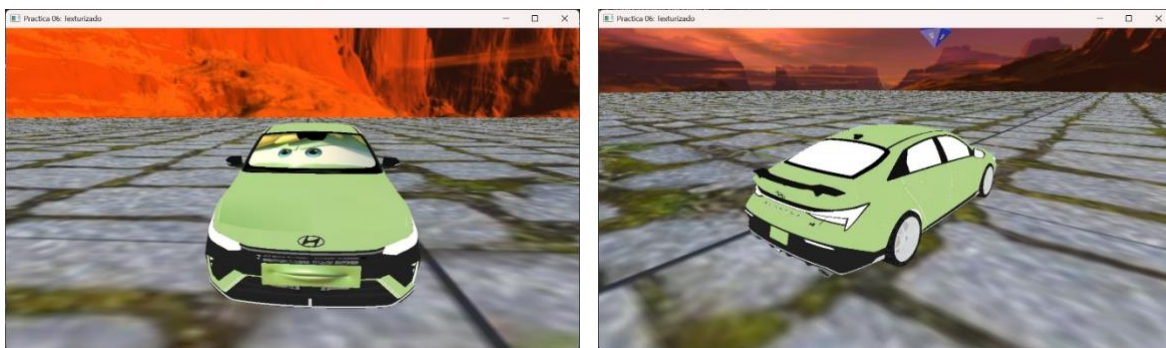
Adicionalmente, separe el modelo del portaplacas para poderle colocar el modelo de la hoja del dibujo. Para este simplemente fue necesario rotar un poco los vértices y ajustarlos a la posición adecuada como se puede visualizar a continuación:



Ya dentro dentro del código, este se instanció de la misma forma que los modelos de las llantas. En este caso, el carro es el modo padre de las llantas, por lo que la variable auxiliar se encuentra declarada después de la primera transformación, la cual es la posición inicial. El fragmento de código generado es el

```
//Instancia del coche
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getmuevex() , -0.5f, -3.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Elantra_M.RenderModel();
```

Al ejecutar el código, el carro ya con las texturas se ve de la siguiente forma:



Problemas presentados

No presente problemas al momento de realizar la práctica debido a que el procedimiento fue bastante similar al del ejercicio previo ya que en el este aprendimos a por medio de código y por medio de 3ds Max. Si bien nunca habíamos modelado una figura distinta a las ya dadas por el profesor, la figura usada para el dado fue bastante sencilla y no me costó trabajo obtener los vértices. Adicionalmente, no conté con ningún problema al momento de compilar y ejecuta el código.

Conclusión

La realización de esta práctica me fue de gran utilidad para aprender las bases del manejo del programa de edición de imágenes GIMP, el cual fue utilizado para modificar las imágenes que se utilizarían como texturas. Además, aprendí a texturizar dentro de 3ds Max y pude seguir practicando con las herramientas de atraco y dettach.

Considero que la explicación del profesor fue adecuada y el documento del manual de prácticas me fue bastante útil para aprender a hacer los materiales en el *material editor*. Para próximas prácticas, mi única recomendación avisar con más tiempo de anticipación a los alumnos que deben visualizar los videos subidos a la plataforma

antes de la clase de laboratorio para poder llevar a cabo las actividades con más fluidez y facilidad.

Bibliografía

Aris Aris. (2018, 30 agosto). *Venta De Llantas*. Pinterest. <https://pin.it/2FifKDRzu>

bemphotos. (s. f.). *Elegante fondo panorámico plata acero metal textura - vector Vector pro*. Vecteezy. <https://es.vecteezy.com/arte-vectorial/7408014-elegante-panoramica-fondo-plata-acero-metal-textura-vector>

Peter Hermes Furian. (s. f.). *Icosahedral dice*. Shutterstock. <https://thumbs.dreamstime.com/b/icosahedral-dice-platonic-solid-template-icosahedron-template-icosahedral-dice-one-five-platonic-solids-make-d-item-118533768.jpg>